

# معماری سیستم فاز اول Unixsee

داشبورد Next.js + لایه BFF با NestJS + پلتفرم عملیاتی WordPress

وضعیت: پیشنهادی

تاریخ: ۲۷ ژوئیه ۲۰۲۶

**تصمیم معماری:** Next.js فقط با NestJS ارتباط برقرار می‌کند. WordPress تجربه پزل مدیریت و فرایندهای کسب‌وکار/ محتوا را فراهم می‌کند. NestJS همچنان درگاه اصلی برنامه، مرز مجزده، بک‌اند عملیاتی و سامانه بلادرنگ باقی می‌ماند.

## ۱. هدف و محدوده

این سند معماری پیشنهادی فاز اول اپلیکیشن Unixsee را بر اساس مسیرهای پیاده‌سازی شده فعلی در داشبورد و جریان‌های محصول مورد توافق تعریف می‌کند. تمرکز سند بر مرزهای سیستم، مالکیت داده، یکپارچه‌سازی‌ها، جریان‌های زمان اجرا و ترتیب پیاده‌سازی است، نه طراحی رابط کاربری یا جزئیات سطح پایین کد.

سورس فعلی Next.js این مازول‌های داشبورد را نمایش می‌دهد:

- نمای کلی داشبورد شامل فهرست وب‌سایت‌ها، اعلان‌ها و فعالیت‌ها.
- پلن‌ها و یک فرم شبیه پرداخت که باید به جریان درخواست پلن تبدیل شود.
- فهرست وب‌سایت‌ها و جزئیات هر وب‌سایت.
- تیکت‌ها و گفت‌وگوهای تیکت.
- درخواست خدمات تکمیلی و پیگیری خدمات.
- پروفایل و تنظیمات امنیتی.
- موضوعات و مقالات مرکز راهنما.
- رابط دامنه‌ها که تا زمان تأیید صریح مالکیت و فرایندهای آن، باید خارج از محدوده قطعی بک‌اند فاز اول باقی بماند.

**یادداشت درباره وضعیت سورس:** رابط فعلی با داده‌های ثابت و آزمایشی کار می‌کند. این معماری مشخص می‌کند مدل‌های نمایشی فعلی چگونه باید توسط سرویس‌های واقعی تأمین شوند، بدون اینکه Next.js مستقیماً به WordPress یا سیستم‌های زیرساختی وابسته شود.

## ۲. جمع‌بندی معماری

Unixsee باید از یک معماری مازولار و لایه‌ای با یک بک‌اند واحد در مقابل فرانت‌اند استفاده کند:

- Next.js لایه نمایش است.
- NestJS نقش Backend-for-Frontend یا BFF، درگاه API، مرجع توکن و احراز هویت، مرز مجزده، هماهنگ‌کننده فرایندها و درگاه بلادرنگ را دارد.

- WordPress پلتفرم عملیات کسب‌وکار و محتوا، شامل تجربه مدیر سیستم، است.
  - PostgreSQL پایگاه داده مرجع برای وبسایت‌ها، سرورها، Agentها، متریک‌ها، عملیات‌ها و رویدادهای فعالیت است.
  - ذخیره‌سازی WordPress برای رکوردهای محتوایی و فرایندهای کسب‌وکارِ سنگین از نظر مدیریت، هرچا اکوسیستم WordPress مزیت روشنی ایجاد کند، مرجع خواهد بود.
  - Edge Agentها فقط با NestJS ارتباط برقرار می‌کنند و هرگز مستقیماً با WordPress یا Next.js در تماس نیستند.
- استفاده از WordPress به‌عنوان پنل مدیریت به این معنا نیست که هر موجودیتی که در WordPress ویرایش می‌شود باید در پایگاه داده WordPress ذخیره شود. افزونه WordPress مربوط به Unixsee باید برای موجودیت‌های عملیاتی تحت مالکیت NestJS نقش رابط Control Plane را داشته باشد و در عین حال مالک بومی فرایندهای محتوایی و کسب‌وکاری مناسب WordPress باشد.

## ۳. معماری منطقی

```

مرورگر کاربر
|
| HTTPS / Socket.IO
v
Next.js داشبورد و رابط عمومی
|
| REST + Socket.IO فقط
v
هسته برنامه / NestJS BFF
|-- Tenant احراز هویت، توکن‌ها و مجوزدهی
|-- API تجميع داشبورد و مدل‌های نمایشی
|-- ها، متریک‌ها، هشدارها و عملیات‌ها Agent، وبسایت‌ها، سرورها
|-- و ردپای ممیزی Unixsee فید فعالیت‌های
|-- WordPress یکپارچه‌سازی Adapter
|
+--> PostgreSQL
+--> Edge Agent ها از طریق HTTPS احراز شده
+--> WordPress Unixsee Bridge API از طریق سرور به‌سرور خصوصی
+--> Media Library وردپرس برای فایل‌های پیوست تیکت و درخواست

WordPress + Unixsee Bridge افزونه پنل مدیریت
|-- مدیریت کاربران و مشتریان
|-- درخواست پلن و فرایند راه‌اندازی
|-- تیکت‌ها و خدمات تکمیلی
|-- اعلان‌ها، مرکز راهنما، استوری‌ها، ارزیابی‌ها و عضویت‌ها
|-- در آینده: صورتحساب، پرداخت، نظرات مشتریان و وبلاگ
+--> WordPress پایگاه داده
+--> برای موجودیت‌های عملیاتی NestJS مدیریت API
  
```

## ۳.۱ Control Plane در برابر Execution Plane

| لایه  | مسئولیت                                                                  | فناوری اصلی |
|-------|--------------------------------------------------------------------------|-------------|
| نمایش | داشبورد مشتری و تجربه عمومی. بدون دسترسی مستقیم به WordPress یا Agentها. | Next.js     |

| لایه                    | مسئولیت                                                                                            | فناوری اصلی                |
|-------------------------|----------------------------------------------------------------------------------------------------|----------------------------|
| برنامه/API              | احراز هویت، مجوزدهی، تجمیع، هماهنگ سازی، REST، Socket.IO و قراردادهای پایدار.                      | NestJS                     |
| Control Plane کسب و کار | صفحات مدیریتی، محتوای ویرایشی، فرایندهای پشتیبانی و خدمات، انتخاب ارائه دهنده و کنترل های عملیاتی. | WordPress + افزونه Unixsee |
| اجرای عملیاتی           | دریافت داده Agent، پایش، ارسال عملیات، ارزیابی هشدار، ترافیک زنده و تولید فعالیت.                  | NestJS + PostgreSQL        |
| داده زیرساخت            | متریک های عملیاتی و بررسی های Append-only ذخیره شده در PostgreSQL.                                 | PostgreSQL                 |

## ۴. مالکیت داده و منبع حقیقت

هر موجودیت باید دقیقاً یک مالک مرجع داشته باشد. ارجاع بین سیستمها مجاز است، اما تکثیر کامل و دوطرفه داده مجاز نیست.

| دامنه / موجودیت                                              | مالک مرجع         | قاعده معماری                                                                                                                                     |
|--------------------------------------------------------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| فهرست کاربران و پروفایل مشتری                                | WordPress         | NestJS فقط شناسه خارجی کاربر WordPress، عضویت Tenant، نمای مجوزها و داده نشست لازم برای اعمال سیاست های API را نگهداری می کند.                   |
| Access/Refresh Token و نشست ها                               | NestJS            | NestJS جریان ورود را اعتبارسنجی می کند، در صورت نیاز مرحله دوم را انجام می دهد، Refresh Token ها را می چرخاند و توکن های Unixsee را صادر می کند. |
| کاتالوگ پلن ها                                               | WordPress         | NestJS یک Read Model نرمال شده برای Next.js ارائه می دهد. قیمت ها و عنوان پلن ها نباید در فرانت اند Hard-code شوند.                              |
| درخواست های پلن و وضعیت راه اندازی                           | WordPress         | این یک فرایند کسب و کاری با تمرکز زیاد بر پلن مدیریت است. NestJS فرمان ها را Proxy کرده و وضعیت را برای داشبورد نرمال می کند.                    |
| سرورها، VPS ها، Agent ها، وبسایت ها و تخصیص سرویس ها         | NestJS/PostgreSQL | صفحات مدیریتی WordPress این رکوردها را از طریق API مدیریتی NestJS مدیریت می کنند؛ WordPress وضعیت کامل آن ها را تکثیر نمی کند.                   |
| دسترس پذیری، بررسی نرم افزار، وضعیت امنیت، ترافیک و متریک ها | NestJS/PostgreSQL | این داده ها توسط Agent ها و Job های بک اند تولید می شوند. فیلدهای زنده از طریق Socket.IO ارسال می شوند.                                          |
| فعالیت های Unixsee                                           | NestJS/PostgreSQL | یک فید واحد برای فعالیت های دستی مدیران و رویدادهای عملیاتی خودکار. WordPress آیتم های دستی را از طریق API مدیریتی NestJS ارسال می کند.          |

| دامنه / موجودیت                                                     | مالک مرجع             | قاعده معماری                                                                                                               |
|---------------------------------------------------------------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------|
| اعلان‌ها/اخبار داشبورد                                              | WordPress             | آیتم‌های محتوایی در WordPress مدیریت و از طریق NestJS تحویل داده می‌شوند.                                                  |
| تیکت‌ها، پیام‌ها و فایل‌های پیوست                                   | WordPress             | NestJS نمای مقابل کاربر است. WordPress فایل‌های پیوست را در Media Library خود ذخیره و به رکوردهای تیکت و پیام متصل می‌کند. |
| درخواست‌ها و تخصیص‌های خدمات تکمیلی                                 | WordPress             | فرایند کسب‌وکار و بررسی مدیر در WordPress انجام می‌شود؛ NestJS یک API پایدار برای مشتری ارائه می‌دهد.                      |
| مرکز راهنما، استوری‌ها، ارزیابی‌ها، عضویت‌ها، نظرات مشتریان و وبلاگ | WordPress             | مدیریت محتوا و سرخ‌ها از مسئولیت‌های بومی WordPress است.                                                                   |
| Job‌های عملیاتی مانند پاک‌سازی کش و بررسی مجدد وضعیت                | NestJS/PostgreSQL     | NestJS عملیات‌ها را مجاز، صف‌بندی، ارسال، ثبت و ممیزی می‌کند.                                                              |
| صورتحساب و پرداخت                                                   | WordPress/WooCommerce | NestJS داده صورتحساب را ارائه می‌دهد و عملیات پرداخت/سفارش را به یکپارچه‌سازی WordPress واگذار می‌کند.                     |

## ۴.۱. مدل مالکیت Tenant و دسترسی

یک Tenant نماینده یک حساب مشتری تأییدشده Unixsee است.

- کاربران از طریق جدول عضویت `tenant_users` به Tenant‌ها متصل می‌شوند.
- هر عضویت، نقش کاربر مانند `owner`، `admin` یا `viewer` را مشخص می‌کند.
- هر وب‌سایت از طریق `websites.tenant_id` دقیقاً به یک Tenant تعلق دارد.
- کاربران مستقیماً به وب‌سایت‌ها متصل نمی‌شوند.
- NestJS با تأیید اینکه کاربر عضو Tenant مالک وب‌سایت است، دسترسی به وب‌سایت را مجاز می‌کند.

وب‌سایت → Tenant → Tenant کاربر → عضویت

مجوزهای کاربر برای وب‌سایت‌های خاص می‌توانند بعداً به‌عنوان یک لایه مجوزدهی جداگانه اضافه شوند، بدون اینکه مدل مالکیت وب‌سایت تغییر کند.

**قاعده حیاتی:** NestJS نباید مستقیماً جدول‌های WordPress را Query کند. ارتباط فقط از طریق یک API خصوصی و نسخه‌بندی‌شده که توسط افزونه Unixsee در WordPress پیاده‌سازی شده انجام می‌شود.

## ۵. مرز ماژول‌های NestJS

| ماژول           | مسئولیت‌های اصلی                                                                                                         |
|-----------------|--------------------------------------------------------------------------------------------------------------------------|
| Auth & Identity | هم‌هنگ‌سازی ورود، Access/Refresh Token، مرحله دوم احراز هویت، لغو نشست، نگاشت کاربر WordPress و Context مربوط به Tenant. |

| ماژول                            | مسئولیت‌های اصلی                                                                                             |
|----------------------------------|--------------------------------------------------------------------------------------------------------------|
| Dashboard                        | تجمیع خلاصه وبسایت‌ها، اعلان‌ها، فعالیت‌های Unixsee و شمارنده‌های مرتبط در یک پاسخ داشبورد.                  |
| Plan Requests                    | نرمال‌سازی کاتالوگ پلن و وضعیت درخواست؛ واگذاری ساخت درخواست و جریان مدیریتی به WordPress.                   |
| Websites & Services              | هویت وبسایت، تخصیص پلن، متادیتای سرویس، لینک‌ها، نمای صورتحساب، مالکیت و مجوزدهی.                            |
| Agents & Ingestion               | دریافت Payload احرازشده Agent، اعتبارسنجی، Idempotency، ذخیره‌سازی و انتشار رویداد.                          |
| Monitoring & Alerts              | وضعیت دسترس‌پذیری، بررسی نرم‌افزار/به‌روزرسانی، وضعیت امنیت، ترافیک، آستانه‌ها، رخدادها و مدیریت داده قدیمی. |
| Realtime Gateway                 | Room های Socket.IO محدودشده به Tenant و به‌روزرسانی‌های زنده وبسایت.                                         |
| Actions                          | پاک‌سازی کش، بررسی مجدد وضعیت، فرمان‌های عملیاتی آینده، وضعیت Job، Retry و رویدادهای ممیزی.                  |
| Activities                       | فید فعالیت دستی و خودکار وبسایت؛ فیلتر بر اساس Tenant و وبسایت.                                              |
| WordPress Integration            | Client دارای Type، Retry، Circuit Breaker، تبدیل Schema، مدیریت Webhook و بی‌اعتبارسازی Cache.               |
| Tickets & Complementary Services | نمای BFF برای فرایندهای پشتیبانی و خدمات تحت مالکیت WordPress.                                               |
| Content & Leads                  | نمای BFF برای مرکز راهنما، اعلان‌ها، استوری‌ها، درخواست ارزیابی، عضویت‌ها، نظرات مشتریان و وبلاگ.            |
| Audit & Administration           | فرمان‌های مدیریتی، ردیابی ممیزی تغییرناپذیر، هویت عامل و تاریخچه تغییرات.                                    |

## ۶. جریان‌های اصلی سیستم

### ۶.۱. درخواست پلن و راه‌اندازی وبسایت

1. کاربر یک پلن را در Next.js انتخاب می‌کند. صفحه فعلی شبیه Checkout باید به «درخواست پلن» تغییر نام و متن داده شود و نباید مفهوم پرداخت را القا کند.
2. Next.js درخواست را همراه با پلن انتخاب‌شده و اطلاعات تماس/وبسایت به NestJS ارسال می‌کند.
3. NestJS، Tenant را اعتبارسنجی کرده و درخواست را به WordPress Unixsee Bridge منتقل می‌کند.
4. WordPress درخواست را ایجاد کرده و برای ارزیابی و پیگیری در اختیار مدیران قرار می‌دهد.
5. پس از تأیید، مدیران زیرساخت را تأمین یا انتخاب می‌کنند. رکوردهای عملیاتی سرور و وبسایت از طریق API مدیریتی NestJS ساخته می‌شوند.

6. فرایند مهاجرت یا ساخت وبسایت خارج از رابط مشتری ادامه پیدا می‌کند و نقاط عطف تأییدشده می‌توانند رویداد فعالیت Unixsee ایجاد کنند.

7. زمانی که وبسایت فعال و به Tenant کاربر تخصیص داده شود، به صورت خودکار در داشبورد نمایش داده می‌شود.

## ۶.۲. نمای کلی داشبورد

1. Next.js یک مدل نمایشی واحد برای داشبورد از NestJS درخواست می‌کند.
2. NestJS خلاصه وبسایت‌ها و فعالیت‌های اخیر را از PostgreSQL می‌خواند.
3. NestJS اعلان‌های منتشرشده را از WordPress، ترجیحاً از Cache کوتاه‌مدت، می‌خواند.
4. NestJS یک پاسخ واحد، فیلترشده بر اساس Tenant و متناسب با کامپوننت‌های پیاده‌سازی‌شده داشبورد باز می‌گرداند.
5. پس از Hydration اولیه REST، فقط فیلدهای زنده از طریق Socket.IO به‌روزرسانی می‌شوند.

## ۶.۳. جزئیات وبسایت و عملیات‌های اجرایی

داده اولیه صفحه از طریق REST بازگردانده می‌شود. فیلدهای زیر زنده هستند و باید با Socket.IO به‌روزرسانی شوند: `availability`، `lastCheckedAt`، `activeNow` و `activeNowMeasuredAt`. مقادیر تاریخی یا ساختاری همچنان از REST تأمین می‌شوند.

- **Clear Cache:** Next.js فرمان را به NestJS می‌فرستد؛ NestJS مجوز را بررسی می‌کند، یک Action Job می‌سازد، آن را به Agent یا Integration صحیح ارسال می‌کند، نتیجه را ثبت کرده و یک رویداد فعالیت ایجاد می‌کند.
- **Renew Service:** Next.js فرمان را به NestJS می‌فرستد؛ NestJS عملیات صورتحساب را به WordPress/WooCommerce واگذار کرده و نتیجه نرمال‌شده را برمی‌گرداند.
- همه فرمان‌های عملیاتی به Idempotency Key نیاز دارند و باید یک رکورد ممیزی ایجاد کنند. Idempotency Key یک مقدار یکتا است که همراه فرمان ارسال می‌شود تا در صورت دوبار کلیک کاربر یا Retry شدن درخواست، یک‌اند همان عملیات را دو بار اجرا نکند.

رکورد ممیزی یک لاگ پایگاه داده است که دقیقاً نشان می‌دهد چه اتفاقی افتاده است:

```
عملیات: پاک‌سازی کش
وبسایت: example.com
کاربر: Ali
زمان: 27-07-2026 10:30
نتیجه: موفق
```

## ۶.۴. فعالیت‌های Unixsee

فعالیت‌های Unixsee یک خط زمانی عملیاتی قابل مشاهده برای Tenant هستند، نه یک فید عمومی از نوشته‌های WordPress. هر آیتم باید شامل شناسه، عنوان، زمان، ارجاع به وبسایت در صورت نیاز، عامل/منبع، نوع رویداد، سطح نمایش و متادیتای اختیاری باشد.

- **فعالیت دستی:** مدیر یک آیتم را در پنل مدیریت WordPress ایجاد می‌کند؛ افزونه آن را به API فعالیت‌های NestJS ارسال می‌کند.

- **فعالیت سیستمی:** NestJS پس از عملیات موفق یا رویدادهای عملیاتی مانند تهیه Backup یا پاکسازی Cache، آیتم را خودکار ایجاد می‌کند.
- فعالیتهای Unixsee در صفحه جزئیات وبسایت نمایش داده نمی‌شوند. این قابلیت می‌تواند بعداً به صورت اختیاری اضافه شود.
- فعالیتهای Append-only هستند. اصلاحات باید با ایجاد رویداد جایگزین/اصلاحی انجام شوند، نه با بازنویسی بی‌صدای تاریخچه.

## ۶.۵. تیکت‌ها

1. کاربر یک سرویس را انتخاب می‌کند، در صورت نیاز یک وبسایت را انتخاب می‌کند، موضوع و توضیحات را وارد می‌کند و فایل‌های پیوست را بارگذاری می‌کند.
2. NestJS مالکیت و محدودیت فایل را اعتبارسنجی می‌کند، فایل‌ها را از طریق API خصوصی WordPress منتقل می‌کند و WordPress آن‌ها را در Media Library ذخیره و به تیکت متصل می‌کند.
3. WordPress تخصیص مدیر، پاسخ‌ها و جریان وضعیت را مدیریت می‌کند.
4. NestJS فهرست نرمال‌شده تیکت‌ها، Thread جزئیات و به‌روزرسانی وضعیت را به Next.js برمی‌گرداند.

## ۶.۶. خدمات تکمیلی

1. کاربر درخواست سرویس را از طریق NestJS ثبت می‌کند.
2. WordPress درخواست را ذخیره کرده و جریان بررسی مدیر را فراهم می‌کند.
3. پس از بررسی، مدیر تخصیص سرویس مشتری را همراه با نوع، وبسایت، وضعیت، تاریخ‌ها، سهمیه یا پیشرفت پروژه و تاریخچه فعالیت ایجاد می‌کند.
4. NestJS این تخصیص را در داشبورد ارائه داده و دسترسی Tenant را اعمال می‌کند.

## ۶.۷. پروفایل و تأیید اطلاعات

NestJS باید OTP را ایجاد کند و سپس شماره تلفن و متن پیام OTP را از طریق یک API خصوصی به WordPress ارسال کند. WordPress با استفاده از ارائه‌دهنده یا افزونه SMS انتخاب‌شده، پیام را ارسال می‌کند.

جریان:

```

را ایجاد می‌کند OTP کد NestJS
→ ارسال می‌کند WordPress شماره تلفن و پیام را به NestJS
→ پیامک را ارسال می‌کند WordPress
→ ارسال می‌کند NestJS را به OTP کاربر
→ کد را اعتبارسنجی می‌کند NestJS
  
```

WordPress نباید OTP را اعتبارسنجی کند.

جریان ورود با نام کاربری/رمز عبور:

- ارسال می‌کند NestJS کاربر نام کاربری/ایمیل و رمز عبور را به
- می‌فرستد WordPress خصوصی احراز هویت Endpoint اطلاعات ورود را به NestJS
- رمز عبور را اعتبارسنجی کرده و فعال بودن حساب را بررسی می‌کند WordPress
- را برمی‌گرداند 2FA نقش و وضعیت Tenant، شناسه کاربر، عضویت WordPress
- را آغاز و اعتبارسنجی می‌کند OTP چالش NestJS، فعال باشد 2FA اگر 2
- را صادر می‌کند Unixsee مربوط به Refresh و Access توکن‌های NestJS، پس از احراز هویت موفق

- خواندن و ویرایش پروفایل از NestJS عبور می‌کند و از طریق API خصوصی روی رکورد کاربر WordPress اعمال می‌شود.
- تأیید ایمیل و شماره تلفن جریان‌های صریح با Challenge‌های دارای انقضا و محدودیت نرخ هستند.
- ارائه‌دهنده فعال SMS می‌تواند از Control Plane پنل WordPress انتخاب شود، اما صدور توکن و تغییر وضعیت‌های تأیید همچنان تحت کنترل NestJS باقی می‌ماند.
- تغییر رمز عبور، نشست‌های Refresh Token مرتبط را لغو می‌کند.
- احراز هویت دومرحله‌ای اختیاری پیش از صدور توکن توسط NestJS اعمال می‌شود.

جریان:

- رمز عبور پذیرفته می‌شود
- فعال است 2FA بررسی می‌کند آیا NestJS
  - را ارسال می‌کند 2FA کاربر کد 2
  - کد را اعتبارسنجی می‌کند NestJS
  - را صادر می‌کند Refresh و Access توکن‌های NestJS

پیش از موفقیت NestJS، 2FA فقط می‌تواند یک توکن موقت و کوتاه‌عمر برای Challenge صادر کند، نه توکن‌های عادی Access/Refresh.

## ۶.۸. محتوا، سرنخ‌ها و قابلیت‌های عمومی

| قابلیت          | مالک و الگوی ارائه                                                                                                                  |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------|
| مرکز راهنما     | WordPress موضوعات و مقالات را مدیریت می‌کند؛ API NestJS‌های خواندنی عمومی/مشتري و Cache را ارائه می‌دهد.                            |
| استوری‌های تیم  | WordPress پروفایل‌ها و آیتم‌های تصویری/ویدیویی استوری را مدیریت می‌کند؛ رسانه‌ها در Media Library وردپرس ذخیره می‌شوند.             |
| ارزیابی فروشگاه | فرم عمومی به NestJS ارسال می‌شود؛ NestJS محافظت در برابر سوءاستفاده را اعمال کرده و رکورد Lead/Request را در WordPress می‌سازد.     |
| عضویت خبرنامه   | فرم عمومی از طریق NestJS ارسال می‌شود؛ WordPress رضایت و وضعیت عضویت را ذخیره کرده و با ارائه‌دهنده انتخابی خبرنامه یکپارچه می‌شود. |
| نظرات مشتریان   | فاز بعد؛ WordPress متن، ارتباط با مشتری، رضایت، ویدیو، وضعیت انتشار و ترتیب را مدیریت می‌کند.                                       |
| وبلاگ           | فاز بعد؛ محتوا کاملاً در WordPress نوشته می‌شود و از طریق NestJS یا یک Endpoint کنترل‌شده فقط خواندنی ارائه می‌شود.                 |



## ۷. قراردادهای API و رویداد

### ۷.۱. گروه‌های REST سمت Client

- /auth و /sessions
- /dashboard
- /plans و /plan-requests
- /websites و /websites/:id
- /websites/:id/actions/\*
- /activities و /notifications
- /tickets و /tickets/:id/messages
- /complementary-service-requests و /complementary-services
- /profile و /verification
- /help-center
- /public/assessment ، /public/subscriptions ، /public/stories و در آینده Testimonials/Blog

### ۷.۲. رویدادهای Socket.IO

| رویداد                       | هدف Payload                                                               |
|------------------------------|---------------------------------------------------------------------------|
| website.availability.updated | availability ، lastCheckedAt ، کد اختیاری آخرین بررسی و وضعیت کهنگی داده. |
| website.traffic.updated      | activeNow و activeNowMeasuredAt.                                          |
| website.action.updated       | پیشرفت و نتیجه پاک‌سازی کش یا سایر عملیات‌های غیرهمزمان وب‌سایت.          |
| activity.created             | درج زنده و اختیاری یک آیتم تازه قابل مشاهده از فعالیت Unixsee.            |

Clientها فقط پس از احراز هویت به Roomهای محدودشده به Tenant و وب‌سایت می‌پیوندند. سرور مجوز Room را از توکن و پایگاه داده استخراج می‌کند؛ Clientها هرگز شناسه Tenant دلخواه را انتخاب نمی‌کنند.

### ۷.۳. قرارداد یکپارچه‌سازی WordPress

- Endpointهای خصوصی و نسخه‌بندی‌شده در افزونه Unixsee Bridge.
- احراز هویت سرور به سرور با درخواست امضاشده یا mTLS و در صورت امکان محدودیت شبکه.
- DTOهای پایدار و مستقل از نام جدول‌های WordPress و Schema افزونه‌های ثالث.
- Idempotency برای ساخت درخواست، پاسخ‌ها، تمییدها و پردازش Webhook.
- Webhookهای خروجی WordPress برای وضعیت کاربر، تغییرات تیکت، تغییرات درخواست پلن، تغییرات خدمات تکمیلی و بی‌اعتبارسازی محتوای منتشرشده.
- Retry با Backoff محدود، Circuit Breaking و مدیریت Dead-letter برای رویدادهای همگام‌سازی ناموفق.

## ۸. الزامات امنیت و چندمستاجری

- Next.js هرگز اعتبارنامه مدیر WordPress، دسترسی پایگاه داده، Secret های Agent یا توکن های زیرساخت را دریافت نمی کند.
- هر درخواست مشتری پیش از اجرای هر Repository Call یا فراخوانی WordPress در NestJS به Tenant محدود می شود.

کاربر وبسایت با شناسه 25 را درخواست می کند  
→ کاربر وارد شده را می خواند Tenant شناسه NestJS  
→ Tenant بررسی می کند که وبسایت 25 متعلق به همان NestJS باشد  
→ را فراخوانی می کند WordPress می گیرد یا PostgreSQL Query از NestJS فقط پس از آن

- Access Token ها کوتاه عمر هستند. Refresh Token ها چرخانده می شوند، در حالت ذخیره شده Hash می شوند، قابل لغو هستند و به رکورد نشست/دستگاه متصل می شوند.
- غیرفعال سازی کاربر در WordPress، تغییر نقش، تغییر رمز عبور و حذف حساب باید از طریق همگام سازی مبتنی بر Webhook نشست های NestJS را محدود یا باطل کند.
- دریافت داده Agent از Credential جداگانه هر HMAC، Agent یا روش مشابه برای احراز درخواست، محافظت در برابر Replay با Timestamp/Nonce و اعتبارسنجی Payload استفاده می کند.
- فایل های پیوست تیکت و درخواست دارای محدودیت Content-Type/Size، اسکن بدافزار، ذخیره سازی در Media Library وردپرس و بررسی دسترسی محدود شده به Tenant هستند.
- عملیات های اجرایی به Capability Check، صریح، Idempotency، Rate Limit، ثبت ممیزی و رفتار امن Timeout/Retry نیاز دارند.
- Secret ها خارج از فیلدهای محتوایی WordPress و سورس کد ذخیره می شوند. رابط مدیر می تواند ارائه دهنده را انتخاب کند، اما Secret باید در Secret Store رمزگذاری شده یا تنظیمات محافظت شده زمان اجرا نگهداری شود.
- هر تغییر مهم باید یک Audit Log ایجاد کند.

## ۹. قابلیت اطمینان، Cache و کارهای غیرهمزمان

- اگر محتوای WordPress موقتاً در دسترس نباشد، NestJS باید داده ناقص اما قابل استفاده داشبورد را بازگرداند؛ داده عملیاتی وبسایت باید همچنان قابل استفاده بماند.
- داده های کم تغییر WordPress مانند پلن ها، اعلان ها، محتوای مرکز راهنما و استوری های منتشر شده Cache شوند. بی اعتبارسازی از طریق Webhook های WordPress انجام شود.
- دسترس پذیری زنده یا ترافیک فعال نباید طوری Cache شود که گویی داده جاری است. زمان اندازه گیری ذخیره و وضعیت کهنگی داده اعلام شود.
- برای ارسال SMS/Email، اسکن فایل پیوست، Retry کردن Webhook وردپرس، عملیات وبسایت و در آینده تطبیق صورت حساب از Background Job استفاده شود.
- در فاز اول می توان از یک Instance واحد NestJS همراه با Event Dispatcher انتزاعی استفاده کرد. Redis/BullMQ و Adapter ردیس Socket.IO زمانی اضافه شوند که مقیاس افقی یا حجم ارسال آن ها را ضروری کند.

- متریک‌ها در PostgreSQL به صورت Append-only باقی می‌مانند، ستون‌های زمان از نوع Timestamp With Time Zone دارند و از کلید مرکب زمان/موجودیت استفاده می‌کنند.

## ۱۰. ترتیب پیشنهادی پیاده‌سازی

| مرحله                      | خروجی‌ها                                                                                                                                               | شرط خروج                                                                                                          |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| ۱. زیرساخت پایه            | ساختار پروژه NestJS، Schema، پایگاه داده PostgreSQL، مدل احراز هویت/نشست، مجوزدهی Tenant، اسکلت WordPress، Bridge، قواعد API و Audit Log.              | کاربر WordPress بتواند وارد شود و نشست Unixsee محدودشده به Tenant دریافت کند.                                     |
| ۲. راه‌اندازی و وب‌سایت‌ها | کاتالوگ/Read Model، پلن، درخواست پلن، جریان مدیریتی راه‌اندازی، API مدیریتی سرور/وب‌سایت، فهرست و جزئیات وب‌سایت، دریافت داده Agent و Socket.IO اولیه. | یک درخواست تأییدشده بتواند به وب‌سایتی تبدیل شود که برای کاربر درست با دسترس‌پذیری/ترافیک زنده نمایش داده می‌شود. |
| ۳. ارتباطات داشبورد        | تجمع داشبورد، فعالیت‌های Unixsee، اعلان‌های مدیریت‌شده با WordPress، عملیات وب‌سایت و وضعیت عملیات.                                                    | رابط فعلی نمای کلی و وب‌سایت به‌طور کامل با API واقعی کار کند.                                                    |
| ۴. فرایندهای پشتیبانی      | تیکت‌ها، پیام‌ها، فایل‌های پیوست، درخواست و تخصیص خدمات تکمیلی.                                                                                        | کاربران و مدیران بتوانند هر دو فرایند را از ابتدا تا انتها انجام دهند.                                            |
| ۵. حساب و محتوا            | ویرایش پروفایل، تأیید اطلاعات، 2FA اختیاری، مرکز راهنما، استوری‌ها، فرم ارزیابی و عضویت خبرنامه.                                                       | قابلیت‌های فعلی حساب و محتوا دیگر از Fixture استفاده نکنند.                                                       |
| ۶. قابلیت‌های آینده        | صورتحساب/پرداخت، نظرات مشتریان، وبلاگ، Redis/BullMQ و هشداردهی پیشرفته.                                                                                | فقط پس از پایدار شدن مالکیت‌ها و قراردادهای فاز اول معرفی شوند.                                                   |

## ۱۱. تصمیم‌ها و محدودیت‌های معماری

| تصمیم                                                               | دلیل                                                                                                                                  |
|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| NestJS تنها API مورد استفاده Next.js است.                           | از وابستگی فرانت‌اند به Schemaهای WordPress جلوگیری می‌کند، احراز هویت و مجوزدهی را یکپارچه می‌سازد و یک قرارداد پایدار ارائه می‌دهد. |
| WordPress یک پلتفرم مدیریت/کنترل است، نه پایگاه داده عمومی همه‌چیز. | سرعت پیاده‌سازی پیل مدیریت حفظ می‌شود، بدون اینکه بارهای عملیاتی و سری زمانی به WordPress تحمیل شوند.                                 |
| داده عملیاتی وب‌سایت تحت مالکیت NestJS است.                         | Agentها، متریک زنده، عملیات، هشدارها و ارتباطات سرورها به بک‌اند اختصاصی و ذخیره‌سازی رابطه‌ای/سری زمانی نیاز دارند.                  |
| فرایندهای کسب‌وکار/محتوا در موارد مناسب تحت مالکیت WordPress هستند. | تیکت، درخواست، انتشار، صورتحساب و محتوا از پیل مدیریت WordPress و Integrationهای آن سود می‌برند.                                      |
| فعالیت‌ها تحت مالکیت NestJS هستند.                                  | فید باید کار دستی تیم را با رویدادهای خودکار زیرساخت ترکیب کند و به‌صورت پایدار به وب‌سایت و Tenant متصل بماند.                       |

| تصمیم                                              | دلیل                                                                                              |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------|
| یکپارچه‌سازی بین سیستم‌ها مبتنی بر API/رویداد است. | خواندن مستقیم پایگاه داده وابستگی شکننده ایجاد می‌کند و مجوزها و قواعد داخلی سرویس را دور می‌زند. |
| جریان Checkout فعلی به جریان درخواست تبدیل می‌شود. | فرایند واقعی کسب‌وکار با ارزیابی و تأمین زیرساخت آغاز می‌شود، نه پرداخت فوری.                     |

## ۱۲. موارد خارج از هدف فاز اول

- تفکیک به Microservice ها؛ NestJS باید کار را به‌صورت Modular Monolith آغاز کند.
- ارتباط مستقیم Next.js با WordPress.
- ذخیره متریک‌های پرتکرار در WordPress.
- تکثیر کامل رکوردهای کاربر، وب‌سایت، تیکت یا سرویس در هر دو پایگاه داده.
- فرض اینکه چون یک عملیات در پنل WordPress نمایش داده می‌شود، پس خود WordPress نیز باید آن را اجرا کند.
- Checkout کامل پرداخت پیش از پایدار شدن جریان راه‌اندازی و فعال‌سازی سرویس.
- Redis یا چند Instance از NestJS پیش از آنکه مقیاس واقعی آن‌ها را ضروری کند.

## ۱۳. معیارهای پذیرش معماری

- مرورگر برای داده برنامه و به‌روزرسانی‌های بلادرنگ فقط با NestJS ارتباط برقرار کند.
- هر موجودیت یک مالک مرجع مستند داشته باشد.
- اختلال WordPress فقط قابلیت‌های تحت مالکیت WordPress را تضعیف کند و وضعیت عملیاتی وب‌سایت‌های ذخیره‌شده در NestJS را پنهان نکند.
- کاربر نتواند با تغییر یک شناسه به وب‌سایت‌ها، تیکت‌ها، فعالیت‌ها، درخواست‌ها یا فایل‌های پیوست Tenant دیگر دسترسی پیدا کند.
- انتخاب پلن یک درخواست ایجاد کند و هرگز پرداخت را به‌اشتباه تأیید نکند.
- تأمین زیرساخت توسط مدیر، رکوردهای سرور و وب‌سایت تحت مالکیت NestJS را از طریق API مدیریتی احرازشده ایجاد کند.
- فعالیت‌های داشبورد بتوانند هم از مدیران و هم از رویدادهای خودکار سیستم منشأ بگیرند، اما از طریق یک فید واحد NestJS نمایش داده شوند.
- فیلدهای زنده از طریق Room‌های احرازشده Socket.IO به‌روزرسانی شوند، درحالی‌که REST منبع داده اولیه و تاریخی باقی بماند.
- یکپارچه‌سازی WordPress بدون تغییر کامپوننت‌های Next.js یا افشای Schema‌های WordPress برای Client قابل تعویض یا اصلاح باشد.

این پیشنهاد بر پایه موارد زیر تهیه شده است:

- آرشیو سورس فعلی Next.js مربوط به Unixsee و مسیرها/کامپوننت‌های پیاده‌سازی شده داشبورد.
- نیازمندی‌های مورد توافق قابلیت‌ها و جریان‌های کاری در بحث معماری.
- سند موجود «معماری سیستم و راهبرد داده» شامل دریافت داده Agent، راهبرد داده Socket.IO، PostgreSQL، انتشار رویداد، چندمستاجری و هشدار رخدادها.